Unidade III

5 APROFUNDAMENTO NA LINGUAGEM C#

Na elaboração de softwares com a linguagem de programação C#, diversos conceitos e recursos são empregados para criar aplicações robustas, flexíveis e eficientes. Nesse contexto surgem abstrações e estratégias como manuseio adequado de situações imprevistas, delegação de responsabilidades, organização e agrupamento de dados de tipos diferentes, administração eficiente de conjuntos de objetos e manipulação e busca de strings baseadas em padrões.

A gestão apropriada de situações inesperadas é crucial para desenvolver softwares resilientes e confiáveis. Uma abordagem inadequada pode levar a falhas de produto e a uma péssima experiência do usuário. Dessa forma, é essencial o entendimento profundo de como tratar e responder a erros e eventos imprevistos, garantindo que a aplicação continue funcionando de maneira adequada, ou que falhe de modo controlado, fornecendo feedbacks úteis ao usuário.

Além disso, no desenvolvimento orientado a objetos, delegar responsabilidades é fundamental para criar sistemas modularizados e reutilizáveis. Isso envolve a atribuição de tarefas e funções específicas a diferentes objetos e métodos, possibilitando a criação de código mais limpo, organizado e fácil de manter, onde cada componente realiza uma função bem definida, reduzindo a complexidade e facilitando a detecção e correção de erros. Ademais, na composição de dados complexos, frequentemente é necessário agrupar valores de tipos diferentes de maneira estruturada e eficiente. Estruturas que permitem agrupar diferentes tipos de dados em uma única unidade tornam-se indispensáveis, proporcionando uma forma intuitiva e concisa de representar e manipular informações diversificadas.

Outro pilar fundamental na construção de software é a capacidade de gerenciar conjuntos de objetos de maneira otimizada, o que requer estruturas de dados avançadas, que permitam manipular, pesquisar e armazenar dados de forma eficiente, garantindo o acesso rápido e a utilização eficaz de recursos de memória.

A capacidade de manipular e buscar strings de maneira eficaz, baseando-se em padrões específicos, é indispensável em muitas aplicações. Identificar padrões e extrair informações a partir de textos são tarefas comuns, que demandam ferramentas precisas e flexíveis para analisar e processar strings, permitindo implementar funcionalidades complexas de forma mais simplificada. Esse é o objeto das expressões regulares.

5.1 Exceções

Exceções são mecanismos para manipular situações inesperadas ou excepcionais durante a execução de um programa, como tentativas de acessar um índice fora dos limites de uma matriz ou tentativas de dividir por zero. Essas anomalias podem originar falhas no código, condições imprevistas, falhas de recursos ou entradas inválidas do usuário. As exceções em C# são representadas por objetos de classes que derivam da classe-base Exception, que é parte do namespace System.

Quando uma condição excepcional é detectada, uma exceção é gerada, ou "lançada", usando a palavra-chave **throw**, ato que interrompe o fluxo normal de execução do programa e começa a procurar um bloco de código designado para lidar com a exceção, conhecido como **catch**. Blocos catch são definidos dentro de construções **try-catch**, onde o bloco **try** contém o código que pode lançar uma exceção, e o bloco catch possui o código para tratar a exceção, caso ocorra. Se uma exceção não é capturada por um bloco catch, ela propaga-se pela pilha de chamadas, potencialmente alcançando o método Main e, se ainda não for capturada, termina o programa.

```
1.  using System;
2.  class Heroi
3.  {
4.     // Propriedade para armazenar o nome do herói.
5.     public string Nome { get; private set; }
6.     // Construtor da classe que inicializa a propriedade nome.
7.     public Heroi(string nome)
8.     {
9.        if (string.IsNullOrEmpty(nome)) // Verifica se o nome é nulo ou
vazio.
10.       {
11.          throw new ArgumentException("Um herói deve ter um nome!",
nameof(nome)); // Lança uma exceção se o nome é inválido.
12.       }
13.       Nome = nome; // Atribui o nome válido à propriedade Nome.
14.    }
15. }
16. class Programa
17. {
18.    static void Main()
19.    {
20.       try
21.       {
22.          // Tentamos criar um herói sem nome, o que é inválido e vai
lançar uma exceção.
23.          Heroi herói = new Heroi("");
24.       }
25.       catch (ArgumentException ex) // Captura qualquer exceção do tipo
ArgumentException.
26.       {
27.          // Escreve a mensagem de erro no console.
28.          Console.WriteLine($"Erro ao criar um herói: {ex.Message}");
29.       }
30.    }
31. }
```

Figura 142 – Exceções: exemplo de utilização

Para exemplificar o uso de exceções, considere o código-fonte da figura 142, no qual criamos uma classe Heroi e simulamos a criação de um objeto com valor inválido, gerando uma exceção. A classe Heroi possui uma propriedade Nome, e seu construtor exige um nome válido (não nulo e não vazio) para criar um objeto dessa classe. Se tentarmos criar um heroi com nome inválido, uma exceção do tipo ArgumentException é lançada. No método Main da classe Programa, tenta-se criar um objeto da classe Heroi com um nome inválido, dentro de um bloco try, e o bloco catch correspondente captura e trata a exceção lançada, exibindo uma mensagem de erro no console.

Exceções oferecem uma maneira robusta e estruturada de lidar com erros e condições excepcionais, permitindo ao código que trata do erro se separar do código principal, facilitando sua legibilidade, manutenção e depuração. Também possibilitam que o programa responda de forma apropriada quando ocorre um erro, seja liberando recursos, registrando informações de diagnóstico ou notificando o usuário antes de terminar ou continuar a execução.

Observação

É importante utilizar exceções de forma judiciosa, apenas em situações verdadeiramente excepcionais, uma vez que o processo de lidar com exceções pode ser custoso em termos de desempenho. Se condições excepcionais forem esperadas ou puderem ocorrer frequentemente, estratégias alternativas como testes condicionais ou códigos de erro podem ser mais apropriadas.

Além disso, é boa prática criar tipos de exceção personalizados com o objetivo de representar erros específicos do domínio da aplicação, proporcionando manejo mais preciso e informativo das situações excepcionais.

5.2 Delegação

Delegação é um recurso que permite encapsular métodos em objetos, sendo crucial para muitos padrões de design e práticas de programação e frequentemente usado para implementar eventos e callbacks, tornando os programas mais modulares, extensíveis e adaptáveis. A delegação em C# é implementada através do **delegate**, um tipo de referência que pode encapsular um método com um tipo específico de assinatura e retorno. Um delegate pode referenciar tanto métodos estáticos quanto instância e, uma vez criado, pode ser invocado como qualquer outro método. Ao criar um delegate, não se está chamando um método, mas criando uma referência de tipo seguro para esse método; ou seja, o compilador de C# verificará se o método referenciado pelo delegate tem a assinatura correta, garantindo consistência e segurança do tipo na hora da compilação.

Delegação é fundamental para implementar eventos e callbacks em C#, pois permite passar métodos como parâmetros, proporcionando flexibilidade para diferentes partes do código poderem responder a eventos ou serem chamadas de volta quando determinadas condições forem satisfeitas. Por exemplo, na programação orientada a eventos, os objetos podem emitir eventos para sinalizar mudanças de estado, e outros objetos podem subscrever esses eventos usando delegates, permitindo que reajam a essas mudanças de maneira desacoplada. Isso facilita o desenvolvimento de sistemas onde diferentes componentes podem interagir entre si de maneira mais limpa e modular, sem necessidade de referências diretas uns aos outros.

```
1.  using System;
2.  // Delegado que representa uma ação executada por um deus.
3.  public delegate void AcaoDeus(string nomeDeus);
4.  class Program
5.  {
6.    static void Main()
7.    {
8.      Deus zeus = new Deus("Zeus");
9.      Deus hera = new Deus("Hera");
10.     // Zeus está assinando a ação de cumprimentar.
11.     zeus.Acao = Cumprimentar;
12.     // Hera está assinando a ação de despedir.
13.     hera.Acao = Despedir;
14.     zeus.ExecutarAcao();
15.     hera.ExecutarAcao();
16.   }
17.   // Método que implementa a ação de cumprimentar.
18.   static void Cumprimentar(string nomeDeus)
19.   {
20.     Console.WriteLine($"{nomeDeus} diz: Saudações, mortais!");
21.   }
22.   // Método que implementa a ação de despedir.
23.   static void Despedir(string nomeDeus)
24.   {
25.     Console.WriteLine($"{nomeDeus} diz: Até mais, mortais!");
26.   }
27. }
28. class Deus
29. {
30.   public string Nome { get; }
31.   public AcaoDeus Acao { get; set; }
32.   public Deus(string nome)
33.   {
34.     Nome = nome;
35.   }
36.   // Método que executa a ação assinada pelo deus.
37.   public void ExecutarAcao()
38.   {
39.     Acao?.Invoke(Nome);
40.   }
41. }
```

Figura 143 – Delegação: exemplo de utilização

Observe o código da figura: na linha 3 temos um delegate denominado AcaoDeus, e quando esse delegate é definido o compilador C# automaticamente fornece um método Invoke com a mesma assinatura do delegate. Esse método permite a qualquer método que tenha a assinatura correspondente ser chamado por uma instância do delegate. Na classe Deus (linha 28), a propriedade Acao é do tipo AcaoDeus, portanto detém referência a um método que pode ser invocado pelo método Invoke. Na linha 39, ao ser executada, há uma verificação para saber se Acao é null, isto é, se não há nenhum método associado a ele. Essa verificação se faz com o símbolo **?.**, que é o operador de navegação nula.

Se Acao não for null, o método Invoke é chamado, executando o método associado ao delegate. Então, na essência, o delegate age como um tipo seguro que pode conter referências a métodos, e o Invoke gerado automaticamente permite que esses métodos sejam chamados, possibilitando implementar padrões de design como callback e eventos, desacoplando componentes do software e tornando o código mais modular, reutilizável e mantível.

No método Main são criadas instâncias para Zeus e Hera: Zeus assina o método Cumprimentar e Hera assina o método Despedir. Quando o método ExecutarAcao é chamado para cada deus, ele invoca o método assinado, seja cumprimentando ou se despedindo, evidenciando o uso de delegação.

Observação

O uso de delegados permite criar código mais genérico e extensível, onde novas ações podem ser facilmente adicionadas sem a necessidade de modificar as classes existentes.

Delegados em C# também são a base para as expressões lambda (que veremos em detalhes no tópico 7.1) e LINQ (7.2), tecnologias que permitem criar código mais conciso, legível e expressivo ao manipular coleções de dados.

5.3 Tuplas

São tipos de dados que podem armazenar múltiplos elementos, possivelmente de tipos diferentes, em uma única unidade. Antes da introdução das tuplas valorizadas em C# 7.0, elas existiam como tipos de referência, necessitando da inclusão do pacote System.Tuple para ser utilizadas, e em certa medida eram ineficientes e desconfortáveis. Com o lançamento do C# 7.0, o conceito de tuplas foi aprimorado e reformulado para proporcionar uma maneira mais eficiente, sintaticamente agradável e produtiva de representar estruturas de dados leves e imutáveis. As novas tuplas valorizadas são tipos de valor, sendo mais eficientes em termos de desempenho se comparadas com as tuplas de referência anteriores, e podem ser utilizadas de forma integrada, sem necessidade de bibliotecas adicionais.

Tuplas em C# são particularmente úteis quando se deseja retornar mais de um valor de um método sem a necessidade de criar um tipo de classe ou estrutura adicional, o que facilita a escrita de códigos mais concisos e expressivos. Também servem como uma ferramenta valiosa quando se trabalha com conjuntos temporários de dados relacionados, onde a criação de uma estrutura de dados mais formal seria desnecessária e sobrecarregada.

Embora as tuplas sejam imutáveis por natureza – isto é, uma vez atribuídos, os valores das tuplas não podem ser alterados –, o C# 7.0 introduziu tuplas valorizadas que suportam a decomposição, permitindo extrair elementos de uma tupla em variáveis separadas e suportar a atribuição de nomes significativos aos seus elementos, aumentando a legibilidade do código. É crucial, no entanto, ponderar cuidadosamente ao usá-las, especialmente se a semântica e a intenção do código precisarem ser claras e compreensíveis, dado que, ao contrário de classes e estruturas, tuplas não proporcionam um significado

intrínseco através de nomes de tipos e propriedades. Em muitos cenários, particularmente em sistemas de software grandes e complexos, pode ser mais apropriado definir tipos específicos para representar conjuntos de dados relacionados, a fim de manter a clareza e a manutenibilidade do código.

```
1.  using System;
2.
3.  class Programa
4.  {
5.     static void Main()
6.     {
7.        // Cria uma tupla representando um deus grego, contendo nome,
domínio e símbolo.
8.        var zeus = ("Zeus", "Céu e Trovão", "Águia");
9.
10.       // Exibe as informações da tupla na console.
11.       Console.WriteLine($"{zeus.Item1} é o deus do {zeus.Item2} e seu
símbolo é a {zeus.Item3}.");
12.
13.       // Cria uma tupla nomeada, proporcionando maior clareza ao
código.
14.       var hades = (Nome: "Hades", Dominio: "Submundo", Simbolo:
"Capacete da Invisibilidade");
15.
16.       // Exibe as informações da tupla nomeada.
17.       Console.WriteLine($"{hades.Nome} é o deus do {hades.Dominio} e
seu símbolo é o {hades.Simbolo}.");
18.    }
19. }
```

Figura 144 – Tuplas: exemplo de utilização

No exemplo, tuplas representam deuses da mitologia grega, e cada deus é representado por uma tupla contendo seu nome, domínio e símbolo. Na primeira tupla (zeus) os elementos são acessados através de Item1, Item2 e Item3, que são os nomes-padrão dos elementos de uma tupla não nomeada. Essa tupla é do tipo valor e não requer a inclusão de nenhum pacote adicional, mostrando a simplicidade e a eficiência introduzidas no C# 7.0.

Posteriormente, uma tupla nomeada hades é criada, e cada um de seus elementos recebe um nome significativo, proporcionando maior legibilidade ao código. Isso ilustra como as tuplas nomeadas em C# permitem associar nomes aos elementos da tupla, o que é especialmente útil quando a clareza do código é crucial.

Por fim, o código imprime as informações contidas nas tuplas no console. Isso exemplifica como elas podem ser úteis para agrupar dados relacionados de forma concisa e como podem ser utilizadas com outras funcionalidades da linguagem C#, como a interpolação de strings, para criar códigos claros e expressivos.

5.4 Coleções

Uma coleção refere-se a uma classe ou estrutura que permite agrupar múltiplos elementos; estes podem ser variáveis, instâncias de objetos ou até outras coleções, dependendo do tipo de coleção utilizado. Coleções são vitais porque permitem que os programadores manipulem conjuntos de dados de forma mais eficiente e organizada, facilitando a implementação de diversas operações, como ordenação, filtragem, busca, entre outras. O C# oferece várias classes e interfaces de coleções na BCL, agrupadas principalmente nos namespaces System.Collections, System.Collections.Generic, System.Collections.Concurrent e System. Collections.Immutable.

A classe-base de coleções no namespace System.Collections oferece tipos como ArrayList e HashTable, coleções não genéricas que podem armazenar qualquer objeto. No entanto, têm a desvantagem de não serem seguras em termos de tipo, já que podem armazenar qualquer objeto e normalmente requerem conversões de tipo durante o acesso aos elementos.

Por outro lado, o namespace System.Collections.Generic inclui coleções genéricas, como List<T> e Dictionary<TKey, TValue>, mais modernas e seguras em termos de tipo, permitindo definir o tipo de dados que a coleção pode armazenar e oferecendo maior segurança e performance ao evitar conversões de tipo desnecessárias. Quando for necessário acessar coleções a partir de múltiplas threads simultaneamente, o namespace System.Collections.Concurrent oferece coleções como ConcurrentDictionary<TKey, TValue>, otimizadas para garantir a consistência dos dados em ambientes multithreaded.

A coleção BitArray armazena bits (0s e 1s) de maneira eficiente em termos de memória, sendo útil quando é necessário representar e manipular conjuntos de valores booleanos. Já a LinkedList<T> é uma coleção que implementa uma lista duplamente encadeada, sendo particularmente útil quando são necessárias inserções e deleções frequentes de elementos, pois essas operações são mais eficientes se comparadas com listas baseadas em arrays, como List<T>.

A coleção SortedSet<T>, por sua vez, representa uma coleção de elementos não duplicados, armazenados em ordem, útil quando é essencial manter os elementos ordenados, seja por razões de eficiência na busca ou por requisitos específicos de apresentação dos dados. Finalmente, o namespace System.Collections.Immutable oferece coleções que, uma vez criadas, não podem ser modificadas, e qualquer operação que altere a coleção retornará uma nova instância com as alterações, mantendo a original inalterada.

A grande vantagem de usar coleções em C# é que elas fornecem uma maneira dinâmica e flexível de gerenciar conjuntos de dados. Em comparação com arrays, que têm tamanho fixo, coleções podem expandir e contrair dinamicamente à medida que os dados são adicionados ou removidos. Além disso, coleções genéricas oferecem segurança de tipo, o que ajuda a evitar erros em tempo de execução e a melhorar o desempenho.

5.5 Expressões regulares

Proporcionam uma maneira sofisticada e eficiente de processar texto, permitindo executar operações complexas de manipulação de string. Fundamentam-se na teoria das linguagens formais e consistem em sequências de caracteres que formam um padrão, servindo para descrever um conjunto de strings. Com esse padrão é possível identificar, combinar, substituir ou dividir strings de forma precisa e flexível, utilizando regras específicas e estruturadas.

Oriundas do namespace System.Text.RegularExpressions, as expressões regulares em C# são úteis quando se lida com validação de dados, pesquisa e análise de texto, onde padrões como os de e-mails, números de telefone e códigos postais precisam ser identificados dentro de um texto maior, como documentos, logs ou entradas de usuário. Ao se trabalhar com expressões regulares, emprega-se a classe Regex, que oferece uma série de métodos para trabalhar com padrões. Esses métodos permitem executar diferentes tipos de operação, como pesquisa e correspondência, substituição de texto e divisão de strings, utilizando o padrão definido pela expressão regular.

Para utilizar adequadamente o Regex, é crucial compreender a sintaxe específica, que pode envolver uma variedade de caracteres especiais, quantificadores, agrupamentos e âncoras, cada um com significados e propósitos diferentes. O quadro 30 apresenta uma lista com os principais elementos da sintaxe, comuns em expressões regulares. Por exemplo, caracteres especiais como "**.**" podem representar qualquer caractere, enquanto quantificadores como "*****" indicam a ocorrência de zero ou mais do caractere ou grupo anterior. Compreender essa sintaxe é vital para criar padrões eficazes e precisos, que correspondam ao texto desejado.

Quadro 30 – Expressões regulares: exemplos da sintaxe utilizada

Tipo	Descrição
Ponto (.)	Corresponde a qualquer caractere único, com exceção de uma nova linha. É frequentemente utilizado quando o caractere exato é desconhecido ou variável
Asterisco (*)	Busca por zero ou mais ocorrências do caractere ou grupo anterior. Por exemplo, no padrão a* qualquer quantidade de caracteres a será uma correspondência válida, inclusive uma string vazia
Mais (+)	Corresponde a uma ou mais ocorrências do caractere ou grupo anterior. No padrão a+, por exemplo, uma ou mais ocorrências de a são necessárias para uma correspondência
Interrogação (?)	Busca por zero ou uma ocorrência do caractere ou grupo anterior, tornando-o opcional na busca de correspondências
Chaves ({n}, {n,}, {n,m})	Especificam a quantidade exata, mínima ou intervalo de ocorrências do caractere ou grupo anterior. Por exemplo, a{3} busca três ocorrências consecutivas de a, enquanto a{3,} busca três ou mais, e a{3,5} busca entre três e cinco inclusivamente
Caret (^)	Símbolo de âncora que representa o início de uma linha ou string, utilizado para encontrar correspondências no início de uma linha
Cifrão ($)	É outro símbolo de âncora, porém representa o final de uma linha ou string. É utilizado para encontrar correspondências ao final de uma linha
Colchetes ([])	Definem uma classe de caracteres e correspondem a qualquer caractere único dentro dos colchetes. Por exemplo, [ab] corresponde a a ou b
Parênteses (())	Agrupam parte da expressão regular, permitindo aplicar quantificadores a esse grupo de caracteres. Por exemplo, (ab)+ corresponde a uma ou mais ocorrências de ab

Tipo	Descrição
Barra vertical (|)	Opera alternância e funciona como um OU lógico entre padrões ou caracteres. Assim, a|b corresponde a a ou b
Barra invertida (\)	Caractere de escape que permite usar caracteres especiais como literais, por exemplo, para buscar o caractere ponto literalmente, você usaria \\

Saiba mais

Expressões regulares são muito úteis na computação em geral, mas muitos desenvolvedores consideram-nas um tema árduo. O livro a seguir adota uma abordagem lúdica do assunto:

JARGAS, A. *Expressões regulares*: uma abordagem divertida. São Paulo: Novatec, 2016.

Para exemplificar a utilização no C#, observe o código-fonte da figura 145. Uma lista chamada entidadesMitologicas é criada para armazenar strings que representam diferentes entidades da mitologia grega. Em seguida, é definida uma expressão regular armazenada na variável padrao, que corresponde a strings que começam com a letra A. A classe Regex é então utilizada para compilar a expressão regular, e uma nova lista, entidadesCorrespondentes, é declarada a fim de armazenar as entidades que correspondem ao padrão definido. O código então percorre cada entidade na lista entidadesMitologicas e utiliza o método IsMatch da classe Regex para determinar se a entidade corresponde ao padrão da expressão regular; as que corresponderem serão adicionadas à lista entidadesCorrespondentes. Finalmente, o código exibe as entidades da lista entidadesCorrespondentes que satisfazem o padrão da expressão regular.

```
1.  using System;
2.  using System.Collections.Generic;
3.  using System.Text.RegularExpressions;
4.
5.  class ProgramaMitologia
6.  {
7.    static void Main()
8.    {
9.      // Lista de nomes representando entidades da mitologia grega
10.     List<string> entidadesMitologicas = new List<string>
11.     {
12.       "Zeus", "Hera", "Ares", "Afrodite", "Hermes", "Apolo",
"Ártemis", "Hades"
13.     };
14.
15.     // Expressão regular para encontrar entidades cujos nomes começam
com 'A'
16.     string padrao = "^A";
17.
18.     // Regex compilada utilizando o padrão definido
19.     Regex regex = new Regex(padrao);
20.
21.     // Lista para armazenar as entidades que correspondem ao padrão
22.     List<string> entidadesCorrespondentes = new List<string>();
23.
24.     // Varre a lista de entidades mitológicas e verifica quais
correspondem ao padrão da expressão regular
25.     foreach (string entidade in entidadesMitologicas)
26.     {
27.       if (regex.IsMatch(entidade))
28.         entidadesCorrespondentes.Add(entidade);
29.     }
30.
31.     // Exibe as entidades que correspondem ao padrão
32.     Console.WriteLine("Entidades que correspondem ao padrão:");
33.     foreach (string entidade in entidadesCorrespondentes)
34.       Console.WriteLine(entidade);
35.   }
36. }
```

Figura 145 – Expressões regulares: exemplo de utilização

O exemplo demonstra como as expressões regulares em C# podem ser usadas de maneira eficaz para filtrar e encontrar strings que correspondam a um padrão específico, nesse caso entidades da mitologia grega cujos nomes começam com a letra A.

Ao se usar expressões regulares em C#, deve-se ater à eficiência e ao desempenho, uma vez que expressões mal construídas ou excessivamente complexas podem levar a uma elevada utilização de recursos e a prolongar o tempo de execução. Portanto, ao empregar expressões regulares, avalie a necessidade real dessa técnica e, quando utilizada, garanta que os padrões sejam otimizados e testados rigorosamente para assegurar precisão e eficiência.

6 MELHORANDO A PROGRAMAÇÃO

Melhorar a programação em C# envolve uma combinação de práticas recomendadas e uso efetivo de ferramentas e recursos associados. O primeiro passo para isso é sem dúvida uma compreensão profunda da sintaxe e das funcionalidades básicas da linguagem. Como em qualquer disciplina, os fundamentos são cruciais: dominar conceitos fundamentais – como classes, herança, polimorfismo, interfaces e delegação – fornece uma base sólida para construir habilidades mais avançadas.

Além disso, familiaridade com a sintaxe básica e avançada melhora a leitura e a escrita de códigos. À medida que a Microsoft continua a desenvolver e evoluir o C#, novas versões da linguagem são lançadas, trazendo novos recursos e funcionalidades; portanto é essencial os programadores se manterem atualizados com essas mudanças, pois estar ciente das atualizações mais recentes e saber como e quando usá-las pode proporcionar melhorias significativas na eficiência e na qualidade do trabalho.

Mas nunca é suficiente conhecer somente a teoria: a prática deliberada é uma das maneiras mais eficazes de internalizar o conhecimento e transformá-lo em habilidade. Isso significa não apenas codificar regularmente, mas também desafiar-se com projetos e problemas novos e variados. Ao enfrentar e superar desafios, os programadores não apenas aplicam o que sabem, mas também identificam áreas onde podem faltar conhecimento ou compreensão.

Além disso, a qualidade do código é tão importante quanto sua funcionalidade. Adotar práticas recomendadas de programação e design de software – como princípios SOLID (discutidos no tópico 6.2) e padrões de design – pode garantir que o código seja modular, reutilizável e fácil de manter. Ações como escrever testes unitários e adotar uma abordagem de desenvolvimento orientado a testes também podem elevar significativamente a qualidade e a robustez do código.

O ecossistema ao redor do C# e .NET é vasto. Utilizar eficazmente as ferramentas disponíveis, como o Visual Studio, pode acelerar o desenvolvimento e tornar o processo mais suave. Ferramentas de análise estática de código, por exemplo, podem identificar problemas de código antes que se tornem problemas mais graves em produção.

Uma das melhores maneiras de melhorar em qualquer campo é aprender com os outros. Participar de comunidades (como o Stack Overflow), conferências de desenvolvimento e grupos de usuários locais pode trazer novas perspectivas e insights, pois o feedback é inestimável, e os pares podem oferecer soluções para problemas que talvez nunca tenhamos considerado. Aprimorar a programação em C# exige uma combinação de compreender a linguagem em profundidade, manter-se atualizado com as mudanças, praticar regularmente, adotar medidas recomendadas, dominar ferramentas e aprender com a comunidade. Como qualquer habilidade, leva tempo e esforço, mas com dedicação a maestria é alcançável.

6.1 Boas práticas de codificação

À medida que a tecnologia se desenvolve e a complexidade dos sistemas aumenta, torna-se crucial adotar boas práticas de codificação. Conferindo especificamente a linguagem C#, percebemos que seguir essas práticas não somente facilita a leitura e compreensão do código, mas também minimiza os erros e aumenta a eficiência do software.

Um dos primeiros aspectos a considerar quando se fala em boas práticas de codificação é **clareza**. A clareza do código é diretamente proporcional à facilidade de sua manutenção e debugging. Em C#, isso é possível adotando nomes de variáveis, classes e métodos descritivos. Em vez de usar nomes de variáveis (como x ou y), é muito mais proveitoso usar nomes como **velocidade** ou **distancia** – nomes que não somente ajudam o desenvolvedor original, mas também qualquer outro que possa trabalhar no código no futuro.

Outra prática essencial é **utilizar comentários** de maneira eficaz. Em vez de comentar cada linha, o que pode tornar o código poluído, é preferível comentar blocos de código ou funções complexas que possam não ser imediatamente óbvias para quem lê. Isso serve como bússola para guiar desenvolvedores pela complexidade inerente ao software. Além disso, XML comments, uma característica particular do C#, permitem que desenvolvedores documentem diretamente no código, o que facilita a geração automática de documentação.

A figura 146 apresenta um exemplo prático desse uso. Comentários XML não são como comentários tradicionais, que costumam ser ignorados pelo compilador; em vez disso são projetados para ser processados por ferramentas que podem gerar documentação externa ou fornecer dicas de ferramentas no ambiente de desenvolvimento. Um comentário XML começa com /// (três barras) em vez do usual // (duas barras) para comentários de linha única; isso sinaliza para o compilador que o comentário deve ser tratado de maneira especial.

No código da figura 146, OperacoesMitologicas representa uma classe onde operações são feitas com números que simbolizam características de deuses e heróis da mitologia grega. A soma de poderes combina os poderes de Zeus e Athena, a multiplicação junta as forças de dois heróis, e a divisão estabelece uma proporção entre os seguidores de dois deuses.

```
1.  /// <summary>
2.  /// Classe que representa operações mitológicas.
3.  /// </summary>
4.  public class OperacoesMitologicas
5.  {
6.     /// <summary>
7.     /// Calcula a soma de dois poderes mitológicos.
8.     /// </summary>
9.     /// <param name="poderZeus">Poder de Zeus.</param>
10.    /// <param name="poderAthena">Poder de Athena.</param>
11.    /// <returns>O total combinado de poder.</returns>
12.    public int SomaPoderes(int poderZeus, int poderAthena)
13.    {
14.       return poderZeus + poderAthena;
15.    }
16.
17.    /// <summary>
18.    /// Multiplica a força de dois heróis.
19.    /// </summary>
20.    /// <param name="forcaHeracles">Força de Heracles.</param>
21.    /// <param name="forcaAchilles">Força de Achilles.</param>
22.    /// <returns>O total combinado de força.</returns>
23.    public int MultiplicaForcas(int forcaHeracles, int forcaAchilles)
24.    {
25.       return forcaHeracles * forcaAchilles;
26.    }
27.
28.    /// <summary>
29.    /// Divide os seguidores de um deus pelo outro.
30.    /// </summary>
31.    /// <param name="seguidoresApollo">Seguidores de Apollo.</param>
32.    /// <param name="seguidoresArtemis">Seguidores de Artemis.</param>
33.    /// <returns>A razão entre os seguidores dos dois deuses.</returns>
34.    public double DivideSeguidores(int seguidoresApollo, int
seguidoresArtemis)
35.    {
36.       if(seguidoresArtemis == 0)
37.       {
38.          throw new DivideByZeroException("Artemis não tem seguidores
para a divisão!");
39.       }
40.       return (double)seguidoresApollo / seguidoresArtemis;
41.    }
42. }
```

Figura 146 – XML comments: exemplo de utilização

Note que <summary> oferece uma breve descrição de método, classe, propriedade etc., e é comum ver essa tag logo acima das definições de método ou classe. A tag <param> descreve um parâmetro de método ou função, com um atributo "name" que especifica qual parâmetro está sendo descrito; e a tag <returns> descreve o valor retornado por um método ou função.

Estas são apenas algumas das tags mais comuns, mas existem muitas outras, incluindo <remarks>, <example> e <exception>, que permitem aos desenvolvedores fornecer uma documentação mais detalhada. Ao usar ferramentas como Doxygen ou Sandcastle, os desenvolvedores podem gerar documentações HTML ou outros formatos a partir desses comentários XML. Além disso, ambientes de desenvolvimento, como o Visual Studio, usam esses comentários para fornecer dicas de ferramentas durante a codificação, fazendo os desenvolvedores entenderem melhor o propósito e o uso de métodos, classes ou propriedades específicas.

Evitar a duplicação desnecessária de código também é importante, o que pode ser feito usando métodos, classes-base e interfaces para reutilizar o código de maneira eficiente. Ao evitar a repetição, reduzimos o potencial de erros, já que qualquer mudança necessária no futuro somente precisará ser feita em um local. Além de evitar a duplicação, a **modularização** do código é extremamente útil. Dividir o código em métodos e classes menores, cada um com sua responsabilidade, permite que os desenvolvedores compreendam, testem e mantenham o código com muito mais facilidade.

A reutilização de código é incentivada em C# através de herança e composição. No entanto, deve-se **preferir a composição à herança**, uma vez que esta pode levar a hierarquias complexas e frágeis, enquanto aquela oferece maior flexibilidade, permitindo que os objetos sejam construídos a partir de pequenos componentes que podem ser facilmente reutilizados e testados isoladamente.

Além disso, tipagem forte é uma das características mais poderosas do C#. Aproveitando-se disso, deve-se sempre preferir **tipos explícitos** em vez de var, exceto quando o tipo é óbvio a partir do contexto imediato, pois isso torna o código mais legível e evita possíveis confusões sobre os tipos de variáveis usadas.

Lembrete

Em relação ao tratamento de erros, utilizar exceções é uma prática comum em C#, conforme vimos no típico 5.1. É importante garantir que os erros sejam tratados de forma adequada, lançando e capturando exceções somente quando necessário e evitando a captura de exceções genéricas. Isso mantém a pilha de chamadas intacta e facilita a identificação da origem de um problema.

A **estilização do código** também desempenha papel fundamental nas boas práticas. Tarefas simples – como manter uma consistência na indentação, usar chaves para delimitar blocos de código e manter uma estrutura consistente em todo o código – podem fazer uma grande diferença na legibilidade. O uso adequado de **convenções de nomenclatura** ainda facilita o entendimento e a manutenção do código.

Lembrete

Classes e propriedades, por exemplo, devem seguir a convenção PascalCase, onde a primeira letra de cada palavra é maiúscula, como em NomeDaClasse ou MinhaPropriedade (como vimos no tópico 2.1). Já as variáveis locais e parâmetros devem seguir a convenção camelCase, iniciando com letras minúsculas e capitalizando a primeira letra de cada palavra subsequente, como em minhaVariavel (como vimos no quadro 6).

Finalmente, mas não menos importante, temos o **teste**. Em C#, o desenvolvimento orientado a testes e os testes unitários, muitas vezes com o uso de frameworks como NUnit ou xUnit, garantem que o código funcione como esperado e que regressões sejam identificadas rapidamente. Boas práticas de codificação em C# combinam clareza, modularização, reutilização, tratamento adequado de erros e testes rigorosos. Adotar essas práticas não apenas melhora a qualidade do código, mas também facilita o trabalho dos desenvolvedores, permitindo que eles se concentrem em solucionar problemas e inovar métodos, em vez de se sobrecarregar com erros evitáveis.

6.2 Princípios SOLID, DRY, KISS, YAGNI e Occam

Um conjunto de princípios e acrônimos desempenha papel fundamental na construção de um software robusto, manutenível e eficiente. Embora originados em contextos diversos, todos convergem para um objetivo comum: criar um código limpo e eficaz. Mergulhando nesse universo do C#, é interessante observar como a linguagem e seus recursos complementam esses princípios.

O acrônimo SOLID representa uma série de cinco princípios fundamentais do design orientado a objetos e programação. Promovidos por Robert C. Martin, destinam-se a melhorar a qualidade e manutenibilidade do código-fonte. O **princípio da responsabilidade única**, representado pela letra **S**, enfatiza que uma classe deve ter apenas uma razão para mudar; no ambiente C#, isso pode ser traduzido em manter nossas classes e métodos concisos, possivelmente usando classes parciais ou organizando-as em namespaces específicos para garantir clareza e coesão. Por exemplo, uma classe que lida com operações de banco de dados em C# não deve também ser responsável por lógica de interface do usuário.

O **princípio aberto-fechado**, simbolizado pela letra **O**, pode ser evidenciado pelo poderoso sistema de interfaces e herança da linguagem. Em vez de modificar uma classe existente, os desenvolvedores são encorajados a estender suas funcionalidades através de herança ou implementação de interfaces, garantindo que o código-base existente permaneça intacto, reduzindo a possibilidade de erro.

A letra **L** refere-se ao **princípio da substituição de Liskov**, que se manifesta ao garantir às subclasses que estendem ou implementam superclasses e interfaces respeitar as expectativas originais. Uma subclasse deve ser intercambiável com sua superclasse sem quebrar a aplicação. Já o **princípio da segregação da interface**, representado pela letra **I**, ganha relevância ao considerarmos a modularidade e flexibilidade que C# oferece. Em vez de sobrecarregar uma classe ou interface com múltiplos métodos,

os desenvolvedores podem criar múltiplas interfaces, cada uma focada em uma funcionalidade específica, tornando o código mais legível e mantendo classes e interfaces enxutas e focadas.

Finalmente, a letra **D** simboliza o **princípio da inversão de dependência**, que advoga pela dependência em abstrações e não em implementações concretas, sendo vivamente expresso em C# através de injeção de dependências. Com frameworks como o ASP.NET Core, os desenvolvedores podem facilmente inverter dependências, tornando o código mais modular e testável. Aplicar adequadamente os princípios SOLID pode ajudar a prevenir muitos problemas comuns enfrentados durante o design de software, incluindo desafios relacionados à refatoração, manutenção e escalabilidade do código. No entanto, é essencial abordar esses princípios com discernimento e adaptá-los às necessidades específicas do projeto.

O princípio DRY, que significa *don't repeat yourself* (não se repita, em tradução livre), é uma das pedras angulares da programação moderna e, embora seja aplicável a qualquer linguagem de programação, sua implementação e significado em contextos específicos – como em C# – podem ter nuances. Ele advoga pela redução da repetição de informações ou lógica em sistemas de software, incentivando desenvolvedores a reutilizar o código e evitar a duplicação desnecessária.

No contexto de C#, uma linguagem amplamente usada e rica em recursos, o princípio DRY se manifesta de várias formas. Por exemplo, a linguagem fornece uma série de mecanismos – como herança, interfaces, delegados e métodos de extensão – que podem evitar a repetição de código. Quando uma lógica é repetida em várias partes de um programa, não somente aumenta a probabilidade de erro – pois qualquer mudança na lógica exige modificações em múltiplos locais –, mas também dificulta o entendimento e a manutenção do código. Consideremos, por exemplo, um código em C# que processa informações de clientes. Se em várias partes do código houver trechos que formatam a apresentação do nome do cliente de maneira específica, e essa lógica precisar ser alterada, o desenvolvedor terá que lembrar de atualizar essa lógica em todos os lugares. Em contrapartida, se o código seguisse o princípio DRY, essa lógica estaria centralizada em um único método ou classe, tornando a alteração muito mais simples e menos propensa a erros.

Além dos benefícios diretos de manutenção e clareza, o DRY em C# pode melhorar o desempenho em certas situações. Ao reutilizar o código e as classes eficientemente, desenvolvedores podem aproveitar o poder da compilação *just in time* (JIT) do .NET para otimizar trechos de código frequentemente usados, resultando em execuções mais rápidas.

O princípio KISS, que significa *keep it simple, stupid* ("deixe simples, idiota", em tradução livre), é uma abordagem filosófica em engenharia e design que prioriza a simplicidade. No reino da programação, especificamente ao considerar sua implementação em C#, o KISS nos convida a abordar problemas e soluções com clareza e direção, evitando uma complexidade desnecessária que pode se tornar um fardo ao longo do tempo.

Em C#, uma linguagem rica e poderosa, pode ser tentador mergulhar nas profundezas de seus recursos avançados, no entanto o KISS nos lembra que a solução mais simples é frequentemente a melhor. Isso não significa evitar a inovação ou as características avançadas da linguagem; denota, ao contrário, empregar essas ferramentas de forma criteriosa e apenas quando elas oferecem valor real. Por exemplo, é possível criar estruturas complexas de herança, usar reflexão extensivamente ou empregar padrões de design

avançados, mas, se uma funcionalidade pode ser alcançada mais diretamente sem essas ferramentas, optar pela simplicidade geralmente resulta em código mais legível, mantível e eficiente; se um simples loop foreach serve ao propósito, talvez não seja necessário introduzir uma expressão LINQ complicada.

Além disso, ao aderir ao princípio KISS em C#, desenvolvedores podem encontrar benefícios tangíveis. Código simples frequentemente se traduz em menos erros, pois há menos locais onde as coisas podem dar errado – o que também torna o processo de depuração mais gerenciável, além de o código simples ser frequentemente mais rápido, pois evita operações desnecessárias ou convoluções. Mas talvez o maior benefício em adotar o KISS, especialmente em um ambiente de desenvolvimento colaborativo, seja a facilidade de comunicação. Quando um código é revisitado – seja por um colega ou pelo próprio desenvolvedor – a simplicidade facilita o entendimento e a extensão do trabalho, tornando todo o ciclo de vida do software mais eficaz.

YAGNI – acrônimo de you aren't gonna need it ("você não vai precisar disso", em tradução livre) – é um princípio de desenvolvimento de software que aconselha programadores a não adicionar funcionalidades até que sejam realmente necessárias. Essa filosofia visa combater a tendência de sobre-engenharia, ou seja, a inclinação para criar sistemas excessivamente complexos baseados em necessidades percebidas do futuro. A implementação do princípio YAGNI no contexto do desenvolvimento em C# revela a importância de equilibrar as capacidades expansivas da linguagem com a disciplina de desenvolvimento enxuto.

C# é uma linguagem versátil, que oferece uma variedade de ferramentas e recursos; e para um desenvolvedor faz sentido explorar essas capacidades e construir uma arquitetura robusta e expansiva, antecipando necessidades futuras. No entanto, YAGNI nos lembra que essa abordagem pode levar a um código desnecessário, aumentando a complexidade e potencialmente introduzindo erros; por exemplo, se um desenvolvedor estiver criando uma aplicação em C# e acreditar que ela pode precisar de múltiplos métodos de autenticação, ele pode ser induzido a implementar vários sistemas de login desde o início. No entanto, se a demanda atual requer apenas uma solução simples, o princípio YAGNI sugere começar apenas com essa solução. Quando surgirem requisitos adicionais, o código pode ser adaptado e expandido conforme necessário.

Adotar YAGNI significa resistir à tentação de usar todas as características avançadas e frameworks disponíveis, a menos que haja necessidade clara e presente. Isso não apenas mantém o código mais limpo e compreensível, mas também reduz o tempo de desenvolvimento e teste, pois desenvolvedores não trabalham em recursos desnecessários. Além disso, o princípio YAGNI não promove negligência nem falta de planejamento; em vez disso enfatiza a importância da adaptabilidade. Em vez de tentar prever e preparar-se para todos os cenários futuros possíveis em C#, é mais prático e eficiente construir soluções para os desafios atuais e adaptar-se à medida que novos requisitos emergem.

Em síntese, YAGNI é uma lembrança crucial para os desenvolvedores, para manter o foco no presente, construindo soluções que atendam às necessidades atuais de forma eficiente e evitando a complexidade desnecessária que pode vir da sobre-engenharia.

A navalha de Occam – muitas vezes simplificada como princípio de Occam – é uma heurística filosófica que sugere que, quando confrontados com várias explicações ou soluções concorrentes para

um problema, deve-se escolher a mais simples, que requeira o menor número de pressupostos. Embora tenha raízes na filosofia e na ciência, essa ideia também encontra relevância no campo da engenharia de software, particularmente na programação. Ao programar em C#, como em qualquer linguagem de programação, é comum sermos apresentados a diversos caminhos possíveis para resolver um problema ou implementar uma funcionalidade. Com o vasto conjunto de ferramentas, bibliotecas e padrões à disposição, pode ser tentador criar soluções complexas ou utilizar abordagens sofisticadas para tarefas que poderiam ser abordadas de forma mais simples. Incorporar o princípio de Occam no desenvolvimento em C# é lembrar-se de que a solução mais direta e descomplicada é frequentemente a mais apropriada.

Ao enfrentar um desafio de programação, pode ser útil perguntar "Qual é a forma mais simples e direta de abordar isso sem introduzir complexidade desnecessária?". Por exemplo, se um desenvolvedor estiver considerando um padrão de design complexo ou uma biblioteca de terceiros para uma tarefa em C#, o princípio de Occam sugere pausar e considerar se uma abordagem mais simples – talvez usando recursos nativos da linguagem ou ferramentas já existentes no projeto – poderia ser igualmente eficaz. Isso não significa que soluções avançadas ou complexas devam ser evitadas a todo custo. Em muitos casos, especialmente ao lidar com problemas intrincados, uma solução mais sofisticada é justificada; no entanto a navalha de Occam serve como lembrete para não complicar desnecessariamente, para não adicionar camadas de abstração nem dependências sem uma justificativa clara.

Ao adotar essa mentalidade em C#, desenvolvedores podem produzir códigos mais limpos, mais fáceis de manter e, muitas vezes, mais eficientes. Além disso, ao reduzir a complexidade, a depuração e o teste tendem a ser mais diretos, uma vez que há menos variáveis e componentes interdependentes a considerar.

KISS, YAGNI e navalha de Occam, à primeira vista, podem parecer conceitos redundantes, pois todos enfatizam a importância da simplicidade e advertem contra a complexidade desnecessária. No entanto, ao examinar suas origens e nuances, percebemos que, embora se sobreponham em muitas áreas, cada um oferece uma perspectiva única e aborda diferentes aspectos do desenvolvimento e tomada de decisão. Enquanto os três princípios compartilham um núcleo comum de simplicidade, surgiram de diferentes contextos e têm diferenças sutis em suas ênfases. KISS busca simplicidade de design; YAGNI evita o excesso no desenvolvimento; e a navalha de Occam escolhe uma entre múltiplas soluções.

No cerne de todos os princípios, há um chamado a clareza, eficiência e intencionalidade na criação de software. C#, com sua rica paleta de recursos, permite que os desenvolvedores apliquem esses princípios com eficácia, resultando em um software robusto e sustentável. Mas, como com todas as ferramentas e princípios, a verdadeira maestria está em saber quando e como aplicá-los da maneira mais apropriada.

6.3 Design patterns

Na engenharia de software, padrões de design têm papel crucial na construção de sistemas robustos, escaláveis e mantidos com facilidade. Essas soluções reutilizáveis, surgidas da necessidade de resolver problemas recorrentes na fase de design e implementação de sistemas, transcendem o simples ato de codificar; elas representam um profundo entendimento de como os sistemas funcionam e como componentes distintos interagem entre si.

Embora os padrões de design não sejam presos a uma linguagem específica, cada linguagem, com suas peculiaridades, pode influenciar como determinado padrão é abordado ou implementado. Em C#, a riqueza das bibliotecas-padrão e as características da linguagem tornam a implementação desses padrões particularmente intuitiva. Ao se aprofundar nos padrões de design, é possível identificar categorias que se formaram ao longo do tempo, e cada uma tem um foco.

O quadro 31 apresenta alguns padrões de design comumente usados no mercado de desenvolvimento de software, chamados criacionais.

Quadro 31– Padrões criacionais

Padrão	Descrição
Singleton	Garante que uma classe tenha apenas uma instância e fornece um ponto de acesso global a ela
Factory Method	Define uma interface para criar um objeto, mas permite que as subclasses decidam qual classe instanciar
Abstract Factory	Fornece uma interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas
Builder	Separa a construção de um objeto complexo de sua representação, de forma que o mesmo processo possa criar diferentes representações
Prototype	Especifica os tipos de objeto a criar usando uma instância prototípica e gera novos objetos copiando esse protótipo

Agora, um exemplo prático do uso dos padrões Singleton e Factory. No vasto panteão da mitologia grega, Zeus destaca-se como o soberano do Olimpo. Se olharmos para Zeus sob a ótica do design de software, podemos visualizá-lo como uma entidade singular, que não deveria ser multiplicada nem replicada. Assim, o padrão Singleton encaixa-se perfeitamente para representar Zeus no nosso código.

No código da figura 147, a seguir, a classe Zeus é projetada para garantir que somente exista uma instância dela em todo o programa. Conseguimos isso com um construtor privado, que impede a criação direta de novas instâncias a partir de qualquer outra classe. A única instância existente é mantida em uma variável estática privada denominada **instancia**, acessada através da propriedade Instância. Se, ao tentar acessar essa propriedade, a instância de Zeus ainda não tiver sido criada, ela será instanciada naquele exato momento. Essa técnica é frequentemente chamada inicialização preguiçosa ou lazy initialization.

Adicionalmente, introduzimos o padrão Factory pela classe FabricaDeDeuses, que oferece um método estático chamado CriarDivindade que, ao ser fornecido com o nome de uma divindade, retorna uma instância correspondente. No exemplo, tratamos apenas de Zeus, mas o esqueleto do código permite adicionar facilmente outras divindades.

Ao empregar o padrão Factory, encapsulamos a lógica de criação das divindades, proporcionando mais adaptabilidade e garantindo ao código que interage com as divindades permanecer desacoplado dos detalhes de sua inicialização.

```
1.  // Definição da classe Zeus, que seguirá o padrão Singleton
2.  public class Zeus
3.  {
4.     // Instância privada estática que armazenará a única instância de
Zeus
5.     private static Zeus instancia;
6.
7.     // Construtor privado para garantir que nenhum outro objeto possa
criar uma instância
8.     private Zeus()
9.     {
10.       Console.WriteLine("Zeus foi instanciado.");
11.    }
12.
13.    // Método público estático para acessar a instância única de Zeus
14.    public static Zeus Instancia
15.    {
16.       get
17.       {
18.          // Se ainda não tivermos uma instância de Zeus, criamos uma
19.          if (instancia == null)
20.          {
21.             instancia = new Zeus();
22.          }
23.
24.          // Retornamos a instância única
25.          return instancia;
26.       }
27.    }
28.
29.    public void OrdenarTrovao()
30.    {
31.       Console.WriteLine("Zeus ordena um trovão!");
32.    }
33. }
34.
35. // Definição da classe FabricaDeDeuses, que seguirá o padrão Factory
36. public class FabricaDeDeuses
37. {
38.    // Método estático para criar uma divindade com base no nome
fornecido
39.    public static object CriarDivindade(string nomeDaDivindade)
40.    {
41.       if (nomeDaDivindade == "Zeus")
42.       {
43.          // Retorna a instância única de Zeus
44.          return Zeus.Instancia;
45.       }
46.       else
47.       {
48.          // Para simplificar, não estamos tratando outras divindades aqui
49.          Console.WriteLine($"A divindade {nomeDaDivindade} não é
suportada.");
50.          return null;
51.       }
52.    }
53. }
54.
```

Figura 147 – Singleton e Factory: exemplo de utilização

A figura 148 destaca o código que é a continuação do código da figura 147. No método Main, a seguir, demonstramos como usar a fábrica para obter a única instância de Zeus e invocar um de seus métodos. Tentamos, posteriormente, criar outra divindade – no caso, Atena –, mas como nossa lógica atual não trata essa divindade, recebemos uma mensagem informando que ela não é suportada.

```
1.  public class Programa
2.  {
3.     public static void Main()
4.     {
5.        // Usando a fábrica para obter a instância de Zeus
6.        Zeus zeus = (Zeus)FabricaDeDeuses.CriarDivindade("Zeus");
7.        zeus?.OrdenarTrovao();
8.
9.        // Tentando criar outra divindade
10.       FabricaDeDeuses.CriarDivindade("Atena");
11.    }
12. }
```

Figura 148 – Método Main para Singleton e Factory

Já os padrões estruturais se preocupam com a maneira como as classes ou objetos se compõem, guiando a formação de estruturas que asseguram a harmonia entre as partes, de modo que a mudança em um componente não resulte na revisão completa do sistema. Isso facilita a integração de componentes, mesmo que tenham sido desenvolvidos de forma isolada.

O quadro 32 exemplifica alguns desses padrões.

Quadro 32 – Padrões estruturais

Padrão	Descrição
Adapter	Permite que classes com interfaces incompatíveis trabalhem juntas. Envolve uma classe existente com uma nova interface
Bridge	Desacopla uma abstração de sua implementação para que as duas possam variar independentemente
Composite	Compõe objetos em estruturas de árvore para representar hierarquias de parte-todo. Permite que clientes tratem objetos individuais e composições de objetos uniformemente
Decorator	Anexa responsabilidades adicionais a um objeto dinamicamente. Oferece uma alternativa flexível à subclasse para estender funcionalidades
Facade	Fornece uma interface unificada para um conjunto de interfaces em um subsistema. Define uma interface de nível mais alto que facilita o uso do subsistema
Flyweight	Usa compartilhamento para suportar eficientemente grandes quantidades de objetos finamente granulados
Proxy	Fornece um substituto ou espaço reservado para outro objeto a fim de controlar o acesso a ele

O código-fonte da figura 149 ilustra o uso dos padrões Adapter e Composite.

```
1.  // Interface representando um deus ou deusa da mitologia grega
2.  public interface IDeus
3.  {
4.     void Invocar();
5.  }
6.
7.  // Classe representando Zeus
8.  public class Zeus : IDeus
9.  {
10.    public void Invocar()
11.    {
12.       Console.WriteLine("Zeus lança um raio!");
13.    }
14. }
15.
16. // Classe representando Poseidon, que tem uma invocação diferente (não
compatível com a interface IDeus)
17. public class Poseidon
18. {
19.    public void ConvocarOnda()
20.    {
21.       Console.WriteLine("Poseidon faz as ondas se levantarem!");
22.    }
23. }
24.
25. // Adapter para tornar Poseidon compatível com a interface IDeus
26. public class PoseidonAdapter : IDeus
27. {
28.    private Poseidon poseidon;
29.
30.    public PoseidonAdapter(Poseidon poseidon)
31.    {
32.       this.poseidon = poseidon;
33.    }
34.
35.    public void Invocar()
36.    {
37.       poseidon.ConvocarOnda();
38.    }
39. }
40.
41. // Classe representando um conjunto de deuses (padrão Composite)
42. public class Panteao : IDeus
43. {
44.    private List<IDeus> deuses = new List<IDeus>();
45.
46.    public void AdicionarDeus(IDeus deus)
47.    {
48.       deuses.Add(deus);
49.    }
50.
51.    public void Invocar()
52.    {
53.       foreach (var deus in deuses)
54.       {
55.          deus.Invocar();
56.       }
57.    }
58. }
```

Figura 149 – Adapter e Composite: exemplo de utilização

Inicialmente, temos uma interface chamada IDeus (linha 2) que define um método Invocar, representando a manifestação do poder de um deus. Zeus, rei dos deuses, implementa diretamente essa interface e, ao ser invocado, lança um raio. A intrigante figura de Poseidon, deus dos mares, apresenta um desafio. Seu método de invocação, ConvocarOnda, difere do padrão que estabelecemos com a interface IDeus. Para garantir que Poseidon interaja harmoniosamente com o resto do sistema, sem alterar sua implementação original, utilizamos o padrão Adapter. O PoseidonAdapter permite que Poseidon seja percebido como IDeus, convertendo a chamada de Invocar para ConvocarOnda.

Continuando, representamos o conceito de um panteão – um conjunto de deidades – com o padrão Composite. A classe Panteao também implementa IDeus e tem uma lista de deuses. Ao invocar o panteão, na verdade invocamos todos os deuses que pertencem a ele.

No método Main (figura 150) criamos instâncias de Zeus e Poseidon, e utilizamos o adaptador para integrar Poseidon ao sistema. Posteriormente, estabelecemos um panteão e adicionamos ambas as divindades a ele. Ao invocar o panteão, tanto Zeus quanto Poseidon mostram seus poderes, demonstrando a eficácia e elegância dos padrões Adapter e Composite para resolver desafios de design.

```
1.  public class Programa
2.  {
3.    public static void Main()
4.    {
5.      // Criando deuses individuais
6.      Zeus zeus = new Zeus();
7.      Poseidon poseidon = new Poseidon();
8.
9.      // Adaptando Poseidon para que ele se ajuste à interface IDeus
10.     PoseidonAdapter poseidonAdaptado = new PoseidonAdapter(poseidon);
11.
12.     // Criando um panteão e adicionando deuses a ele
13.     Panteao panteao = new Panteao();
14.     panteao.AdicionarDeus(zeus);
15.     panteao.AdicionarDeus(poseidonAdaptado);
16.
17.     // Invocando todos os deuses do panteão
18.     panteao.Invocar();
19.   }
20. }
21.
```

Figura 150 – Método Main para Adapter e Composite

Por último, mas não menos importante, os padrões comportamentais enfocam a interação e comunicação entre objetos. Esses padrões dão ênfase a algoritmos, processos e à distribuição de responsabilidades entre objetos, assegurando que a colaboração entre eles seja eficaz e coesa. Em C#, padrões como Observer e Strategy são frequentemente empregados para facilitar essa comunicação.

No mundo das aplicações, o Observer é fundamentalmente uma simulação do conceito de assinaturas da vida cotidiana. Assim como uma pessoa pode assinar um jornal e esperar receber notícias atualizadas regularmente, no mundo do software, o Observer permite que um objeto (o observador) "assine" outro para receber atualizações sobre mudanças.

O quadro 33 enumera e descreve diversos padrões comportamentais úteis.

Quadro 33 – Padrões comportamentais

Padrão	Descrição
Chain of Responsibility	Desacopla o remetente da solicitação de seus receptores, permitindo que mais de um objeto manipule a solicitação. Encaminha a solicitação ao longo da cadeia até que um objeto a manipule
Command	Encapsula uma solicitação como um objeto, permitindo que usuários parametrizem clientes com diferentes solicitações, enfileirem solicitações e suportem operações reversíveis
Interpreter	Fornece uma representação para sua gramática e um intérprete para interpretá-la
Iterator	Fornece uma maneira de acessar elementos de uma coleção de objetos sequencialmente sem expor sua representação subjacente
Mediator	Define um objeto que encapsula como um conjunto de objetos interage. O Mediator promove o acoplamento fraco, evitando que os objetos se refiram explicitamente uns aos outros
Memento	Captura e externaliza o estado interno de um objeto sem violar o encapsulamento, a fim de que ele possa ser restaurado para esse estado mais tarde
Observer	Define uma dependência um-para-muitos entre objetos, de modo que, quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente
State	Permite que um objeto altere seu comportamento quando seu estado interno muda. O objeto parecerá ter mudado de classe
Strategy	Define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. A estratégia permite que o algoritmo varie independentemente dos clientes que o utilizam

Para exemplificar esses dois padrões, começamos a figura 151 definindo as interfaces IDeusObserver e IPoderDivino, que representam respectivamente os padrões Observer e Strategy. Já Zeus, deus do trovão, é o personagem central do código e pode ser notificado sobre eventos (padrão Observer) e invocar diferentes poderes divinos (padrão Strategy).

É possível mudar o poder de Zeus em tempo de execução, demonstrando a flexibilidade do padrão Strategy. Inicialmente, ele pode lançar raios, mas esse poder pode ser alterado para causar chuva. No coração do Olimpo, temos uma lista de deuses (_deuses) que observam eventos; quando um evento ocorre no Olimpo, todos os deuses observadores são notificados.

```
1.  using System;
2.  using System.Collections.Generic;
3.  // Interface do Observer
4.  public interface IDeusObserver
5.  {
6.     void Notificar(string mensagem);
7.  }
8.  // Interface do Strategy
9.  public interface IPoderDivino
10. {
11.    void Executar();
12. }
13. public class Zeus : IDeusObserver
14. {
15.    private IPoderDivino _poderAtual;
16.    public Zeus(IPoderDivino poderInicial)
17.    {
18.       _poderAtual = poderInicial;
19.    }
20.    public void DefinirPoder(IPoderDivino novoPoder)
21.    {
22.       _poderAtual = novoPoder;
23.    }
24.    public void InvocarPoder()
25.    {
26.       _poderAtual.Executar();
27.    }
28.    public void Notificar(string mensagem)
29.    {
30.       Console.WriteLine($"Zeus foi notificado: {mensagem}");
31.    }
32. }
33. public class Raio : IPoderDivino
34. {
35.    public void Executar()
36.    {
37.       Console.WriteLine("Lançando um raio!");
38.    }
39. }
40. public class Chuva : IPoderDivino
41. {
42.    public void Executar()
43.    {
44.       Console.WriteLine("Fazendo chover!");
45.    }
46. }
47. public class Olimpo
48. {
49.    private List<IDeusObserver> _deuses = new List<IDeusObserver>();
50.    public void AdicionarDeus(IDeusObserver deus)
51.    {
52.       _deuses.Add(deus);
53.    }
54.    public void EventoNoOlimpo(string evento)
55.    {
56.       foreach (var deus in _deuses)
57.       {
58.          deus.Notificar(evento);
59.       }
60.    }
61. }
```

Figura 151 – Observer e Strategy: exemplo de utilização

O programa da figura 152 começa criando uma instância de Zeus com seu poder inicial, definido como Raio, e Zeus é adicionado ao Olimpo para observar eventos. Ele invoca seu poder para lançar um raio e, em seguida, muda seu poder para Chuva, demonstrando novamente a natureza dinâmica do padrão Strategy. Finalmente, um evento é desencadeado no Olimpo, e Zeus, sendo um observador, é notificado.

```
1.  public class Program
2.  {
3.    public static void Main()
4.    {
5.      Zeus zeus = new Zeus(new Raio());
6.
7.      Olimpo olimpo = new Olimpo();
8.      olimpo.AdicionarDeus(zeus);
9.
10.     zeus.InvocarPoder();
11.
12.     zeus.DefinirPoder(new Chuva());
13.     zeus.InvocarPoder();
14.
15.     olimpo.EventoNoOlimpo("Um grande banquete está sendo preparado no
Olimpo.");
16.   }
17. }
```

Figura 152 – Método Main para Observer e Strategy

Saiba mais

Para aprofundar seu conhecimento em design patterns no contexto de desenvolvimento de software, o livro a seguir é uma excelente referência. É a tradução em português do renomado *Design patterns: elements of reusable object-oriented software*:

GAMMA, E. *et al. Padrões de projeto*: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

Resumo

Tratar exceções em C# é fundamental para criar aplicativos resilientes e tolerantes a falhas, permitindo continuar a execução do programa mesmo diante de erros inesperados. Ao utilizar estruturas de try-catch, o desenvolvedor é capaz de gerenciar falhas de maneira mais refinada, podendo informar ao usuário detalhes específicos do problema ou até mesmo realizar procedimentos de recuperação. Já a delegação permite que métodos sejam passados como parâmetros, o que separa responsabilidades e aumenta a modularidade do código. Esse conceito, quando associado aos princípios SOLID, permite criar sistemas mais coesos e menos acoplados. Os princípios SOLID – acrônimo de *single responsibility*, *open-closed*, *Liskov substitution*, *interface segregation* e *dependency inversion* – guiam o desenvolvimento para uma arquitetura mais limpa e sustentável.

Tuplas são uma adição importante à linguagem C# e auxiliam desenvolvedores a agrupar diferentes tipos de dados de forma imutável e ordenada, promovendo uma estruturação de dados mais limpa e expressiva, especialmente se combinada com coleções que possibilitem armazenar e manipular grupos de objetos, permitindo operações complexas e construindo lógicas avançadas. Nesse contexto, o uso de expressões regulares potencializa a capacidade de manipular strings e validar entradas de dados, contribuindo para a segurança e robustez do sistema. Com elas é possível reconhecer padrões em textos, possibilitando, por exemplo, validar e-mails e outras informações inseridas pelos usuários.

Além disso, ao adotar padrões de design, os desenvolvedores podem resolver problemas recorrentes de maneira mais eficiente e padronizada. Padrões como Singleton podem garantir que uma classe tenha apenas uma instância, enquanto o Observer pode ser empregado para implementar dependências entre objetos de forma desacoplada. Dominar esses elementos é crucial para alcançar excelência na engenharia de software, permitindo a construção de soluções tecnológicas cada vez mais inovadoras e impactantes.

Exercícios

Questão 1. (FGV 2018, adaptada) Na construção de compiladores e no uso de linguagens de programação em geral, expressões regulares constituem um poderoso instrumento para a validação de textos. Nesse contexto, analise a expressão regular exibida a seguir.

```
a{1,4}b*c+
```

Assinale o texto que **não** casa com essa expressão.

A) a

B) ac

C) aacc

D) aaaabcc

E) aabbbc

Resposta correta: alternativa A.

Análise da questão

As expressões regulares proporcionam uma maneira eficiente de processar texto, permitindo a execução de operações complexas de manipulação de strings. As expressões regulares em C# são úteis quando lidamos com validações de dados, pesquisa e análise de texto.

Vamos avaliar, individualmente, as três partes da expressão regular apresentada no enunciado.

- **a{1,4}**: o quantificador designado pelo par de chaves especifica o intervalo de ocorrências do caractere anterior. Logo, no padrão `a{1,4}`, serão buscadas entre uma e quatro ocorrências consecutivas do caractere `a`.

- **b***: o quantificador `*` busca por zero ou mais ocorrências do caractere anterior. No padrão `b*`, qualquer quantidade de caracteres `b` será uma ocorrência válida.

- **c+**: o quantificador + busca por uma ou mais ocorrências do caractere anterior. No padrão `c+`, uma ou mais ocorrências do caractere `c` são necessárias para uma correspondência.

Desse modo, na expressão regular completa, serão consideradas correspondências às strings que apresentam, inicialmente, entre um e quatro caracteres `a`, seguidos de zero ou mais ocorrências do

caractere `b`, seguidas de pelo menos uma ocorrência do caractere `c`. Nas alternativas, a única string que não corresponde a esse padrão é a string `a`, já que ela não apresenta pelo menos uma ocorrência do caractere `c`.

Questão 2. (Instituto AOCP 2018, adaptada) Em relação aos padrões de projeto de software e aos princípios arquiteturais, em POO existe uma série de cinco princípios fundamentais do design e da programação representada pelo acrônimo SOLID. Esses princípios, de acordo com as suas iniciais, são:

A) S (Solid principle) – O (Open principle) – L (Library principle) – I (Integration principle) – D (Double principle).

B) (Security closed principle) – O (Open extend principle) – L (Liskov include principle) – I (Interface duplication principle) – D (Duplicate structure principle).

C) S (Single-closed principle) – O (Open-closed principle) – L (Library exclusive principle) – I (Integration case principle) – D (Dependency inversion principle).

D) S (Security basic principle) – O (Open extern principle) – L (Liskov include principle) – I (Interface duplication principle) – D (Duplicate segregation principle).

E) S (Single responsibility principle) – O (Open/closed principle) – L (Liskov substitution principle) – I (Interface segregation principle) e D (Dependency inversion principle).

Resposta correta: alternativa E.

Análise da questão

Os cinco princípios da POO, identificados pelo acrônimo SOLID, são elencados a seguir.

- **S (single responsibility principle)**: o princípio da responsabilidade única estipula que uma classe deve ter apenas uma razão para ser modificada. Isso significa que devemos alterar uma classe apenas quando os requisitos relacionados à sua responsabilidade específica mudam. Se uma classe tiver várias responsabilidades, cada alteração, em qualquer uma delas, pode afetar outras partes do código. Seguir esse princípio nos ajuda a manter o código-fonte mais organizado e facilita sua manutenção.

- **O (open/closed principle)**: o princípio aberto/fechado indica que as entidades do software (como classes e módulos) devem ser abertas para extensão, mas fechadas para modificação. Isso significa que devemos projetar o código-fonte de forma a permitir que novas funcionalidades sejam adicionadas, sem a necessidade de alterar o código existente. Seguir esse princípio nos ajuda na manutenção, na redução de erros e na reutilização de trechos do código-fonte.

- **L (Liskov substitution principle)**: o princípio da substituição, formulado por Barbara Liskov, sugere que objetos de uma subclasse devem ser capazes de se utilizar no lugar de objetos da sua

superclasse sem afetar o comportamento do programa. Seguir esse princípio nos ajuda a manter a integridade da hierarquia de herança e nos permite utilizar o polimorfismo com segurança.

- **I (interface segregation principle)**: o princípio da segregação de interfaces estabelece que elas devem ter poucos métodos, de modo que as classes que implementam essas interfaces não sejam forçadas a implementar métodos desnecessários. Por isso, as interfaces grandes devem ser "quebradas" em interfaces menores e mais específicas. Seguir esse princípio nos ajuda a manter as classes enxutas, o que contribui com a coesão do código-fonte.

- **D (dependency inversion principle)**: o princípio da inversão de dependência sugere que as classes de alto nível não devem depender diretamente de classes de baixo nível. Em vez disso, ambas as classes (de alto e baixo nível) devem depender de abstrações, como interfaces ou classes abstratas. Seguir esse princípio nos ajuda a reduzir o acoplamento entre as classes e contribui com a flexibilidade do código-fonte.